

Лабораторная работа №12

по курсу: Архитектура компьютера и операционные системы

Иванова Анастасия Сергеевна

2026-04-26

Содержание I

1. Докладчик
2. Вводная часть
3. Выполнение лабораторной работы
4. Контрольные вопросы
5. Вывод

Раздел 1

1. Докладчик

1. Докладчик

- Иванова Анастасия Сергеевна



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25
- Российский университет дружбы народов



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25
- Российский университет дружбы народов
- 1132250427@rudn.ru



Раздел 2

2. Вводная часть

- Командные оболочки являются основным средством взаимодействия с UNIX-подобными операционными системами

- Командные оболочки являются основным средством взаимодействия с UNIX-подобными операционными системами
- Навыки программирования в `bash` необходимы для автоматизации рутинных задач администрирования

- Командные оболочки являются основным средством взаимодействия с UNIX-подобными операционными системами
- Навыки программирования в `bash` необходимы для автоматизации рутинных задач администрирования
- Понимание принципов работы `shell`-скриптов позволяет эффективно управлять системой и обрабатывать данные

- Командные оболочки являются основным средством взаимодействия с UNIX-подобными операционными системами
- Навыки программирования в `bash` необходимы для автоматизации рутинных задач администрирования
- Понимание принципов работы `shell`-скриптов позволяет эффективно управлять системой и обрабатывать данные
- Создание собственных командных файлов значительно ускоряет выполнение повторяющихся операций

Объект исследования: - Командный процессор bash - Управляющие конструкции языка программирования оболочки - Способы обработки аргументов командной строки

Предмет исследования: - Создание и выполнение командных файлов в bash - Использование переменных, массивов и циклов - Обработка параметров и позиционных аргументов - Написание скриптов для автоматизации задач

- Изучить основы программирования в оболочке ОС UNIX/Linux

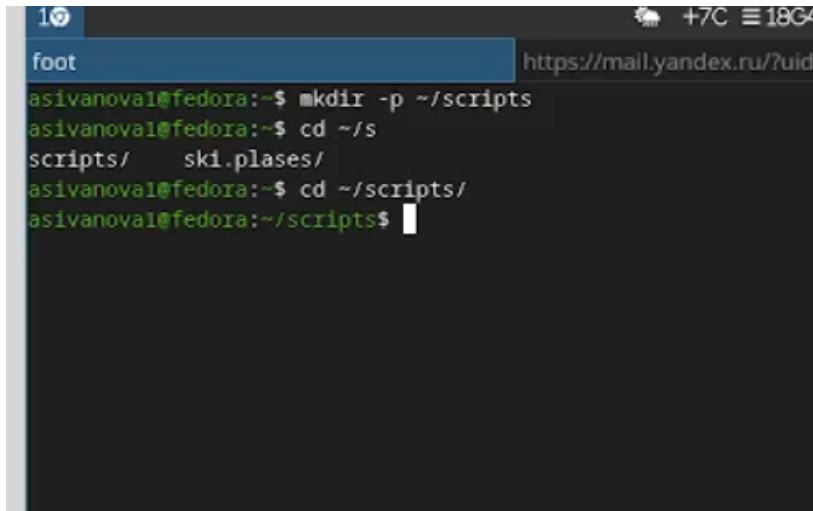
- Изучить основы программирования в оболочке ОС UNIX/Linux
- Научиться писать небольшие командные файлы

Раздел 3

3. Выполнение лабораторной работы

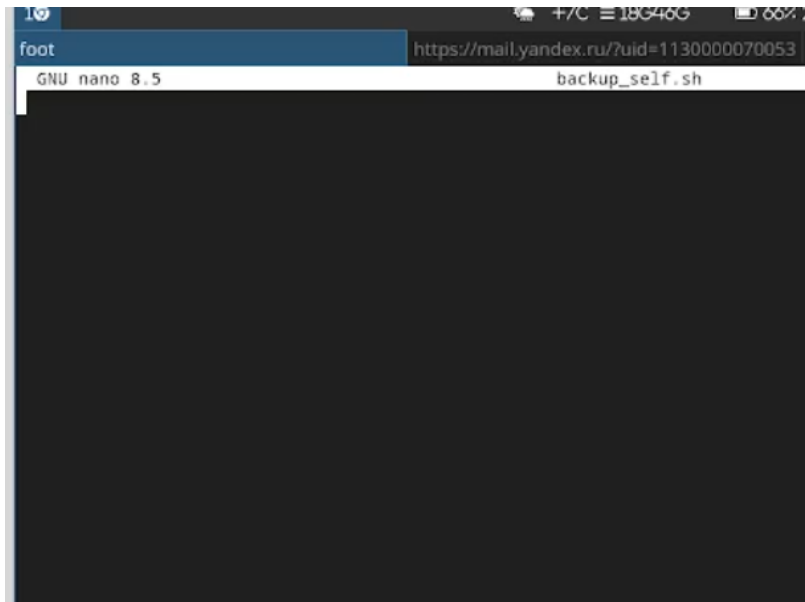
Задание 1. Резервная копия самого скрипта

- 1 Создадим директорию для скриптов (рис. 1):

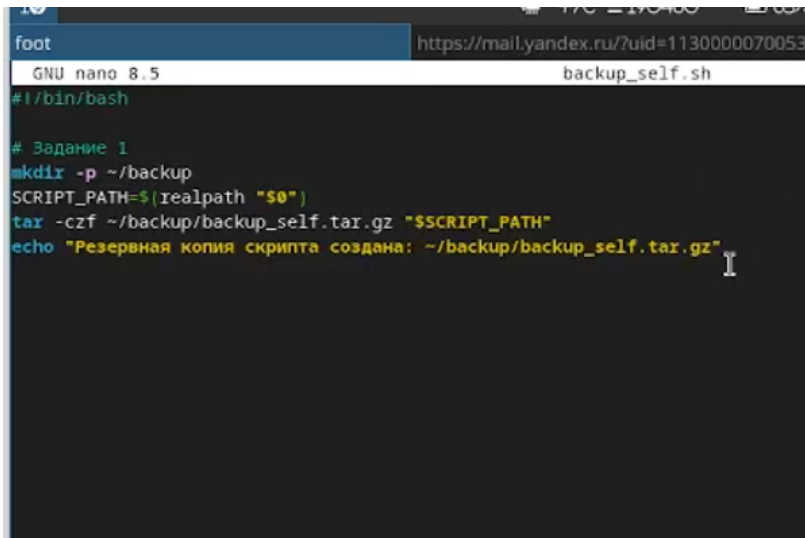
A terminal window on a Fedora system. The user 'asivanova1' is at the prompt. They execute 'mkdir -p ~/scripts', then 'cd ~/s', which shows a directory listing for 'scripts/' and 'ski.plases/'. Then they execute 'cd ~/scripts/' and the prompt changes to '~/scripts\$'.

```
10  
foot  
asivanova1@fedora:~$ mkdir -p ~/scripts  
asivanova1@fedora:~$ cd ~/s  
scripts/      ski.plases/  
asivanova1@fedora:~$ cd ~/scripts/  
asivanova1@fedora:~/scripts$
```

- 2 Создадим файл скрипта `backup_self.sh` (рис. 2):



3 Код скрипта (рис. 3):



```
foot https://mail.yandex.ru/?uid=1130000070053
GNU nano 8.5 backup_self.sh
#!/bin/bash

# Задание 1
mkdir -p ~/backup
SCRIPT_PATH=$(realpath "$0")
tar -czf ~/backup/backup_self.tar.gz "$SCRIPT_PATH"
echo "Резервная копия скрипта создана: ~/backup/backup_self.tar.gz"
```

Рисунок 3: Код скрипта backup_self.sh

- ④ Сделаем скрипт исполняемым (рис. 4):

```
scripts/      ski.plases/  
asivanova1@fedora:~$ cd ~/scripts/  
asivanova1@fedora:~/scripts$ nano backup_self.sh  
asivanova1@fedora:~/scripts$ chmod +x backup_self.
```

Рисунок 4: Изменение прав доступа

- 5 Запустим скрипт (рис. 5):

```
asivanova1@fedora: ~/scripts$ chmod +x backup_self.sh  
asivanova1@fedora:~/scripts$ ./backup_self.sh  
tar: Удаляется начальный '/' из имен объектов  
Резервная копия скрипта создана: ~/backup/backup_self.tar.gz
```

Рисунок 5: Запуск скрипта

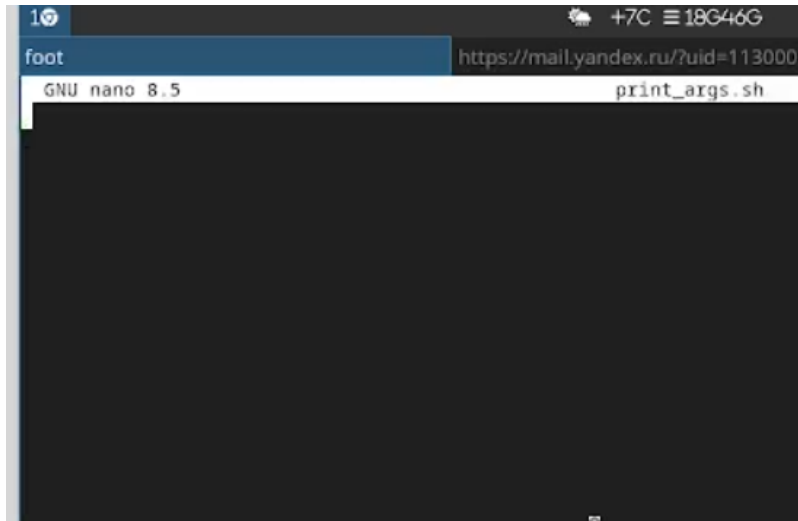
6 Проверим результат (рис. 6):

```
Резервная копия скрипта создана: ~/backup/backup_self.tar.gz
asivanova1@fedora:~/scripts$ ls -la ~/backup/
итого 4
drwxr-xr-x. 1 asivanova1 asivanova1   36 апр 26 18:54 .
drwx-----. 1 asivanova1 asivanova1 1780 апр 26 18:54 ..
-rw-r--r--. 1 asivanova1 asivanova1   315 апр 26 18:54 backup_self.tar.gz
asivanova1@fedora:~/scripts$ tar -tzf ~/backup/backup_self.tar.gz
home/asivanova1/scripts/backup_self.sh
asivanova1@fedora:~/scripts$
```

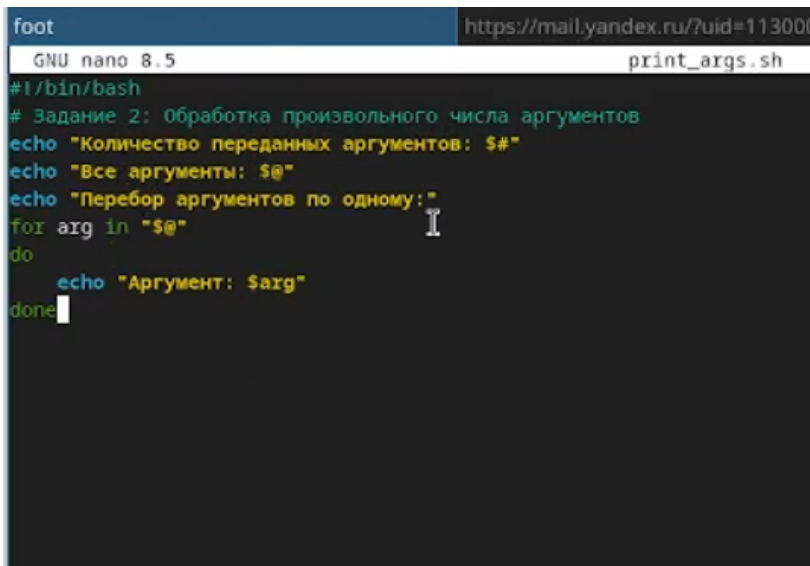
Рисунок 6: Проверка создания резервной копии

Задание 2. Обработка произвольного числа аргументов

- 1 Создадим файл скрипта `print_args.sh` (рис. 7):



2 Код скрипта (рис. 8):



```
foot https://mail.yandex.ru/?uid=113000
GNU nano 8.5 print_args.sh
#!/bin/bash
# Задание 2: Обработка произвольного числа аргументов
echo "Количество переданных аргументов: $#"
```

```
echo "Все аргументы: $@"
echo "Перебор аргументов по одному:"
for arg in "$@"
do
    echo "Аргумент: $arg"
done
```

Рисунок 8: Код скрипта print_args.sh

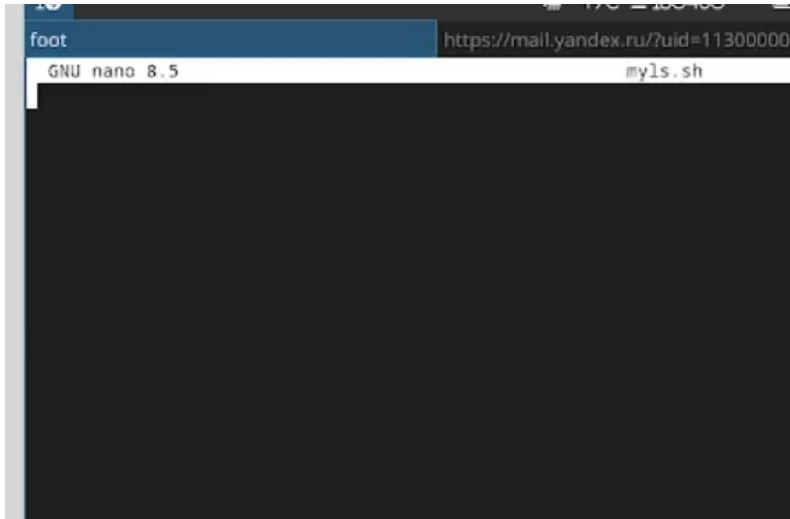
- 3 Запустим скрипт с 8 аргументами (рис. 9):

```
asivanova@fedora:~/scripts$ chmod +x print_args.sh
asivanova@fedora:~/scripts$ ./print_args.sh a b c d e f g h
Количество переданных аргументов: 8
Все аргументы: a b c d e f g h
Перебор аргументов по одному:
Аргумент: a
Аргумент: b
Аргумент: c
Аргумент: d
Аргумент: e
Аргумент: f
Аргумент: g
Аргумент: h
asivanova@fedora:~/scripts$
```

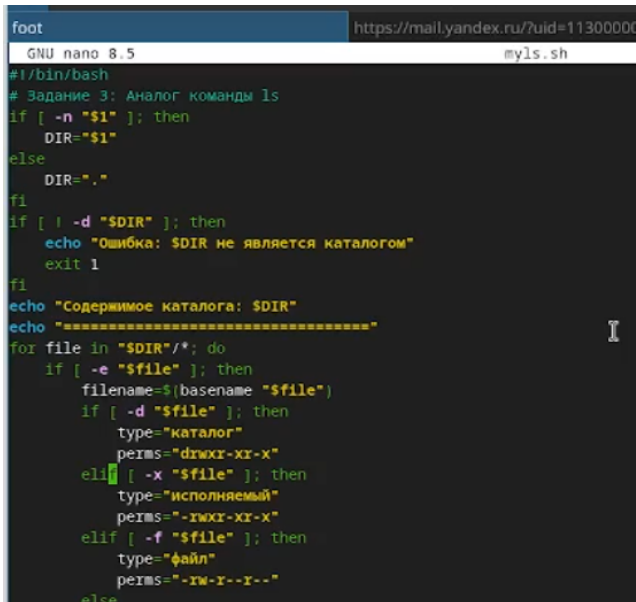
Рисунок 9: Запуск скрипта с 8 аргументами

Задание 3. Аналог команды ls

- 1 Создадим файл скрипта `myls.sh` (рис. 10):



2 Код скрипта (рис. 11):



```
foot https://mail.yandex.ru/?uid=11300000
GNU nano 8.5 myls.sh

#!/bin/bash
# Задание 3: Аналог команды ls
if [ -n "$1" ]; then
    DIR="$1"
else
    DIR="."
fi
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом"
    exit 1
fi
echo "Содержимое каталога: $DIR"
echo "===== "
for file in "$DIR"/*; do
    if [ -e "$file" ]; then
        filename=$(basename "$file")
        if [ -d "$file" ]; then
            type="каталог"
            perms="drwxr-xr-x"
        elif [ -x "$file" ]; then
            type="исполняемый"
            perms="-rwxr-xr-x"
        elif [ -f "$file" ]; then
            type="файл"
            perms="-rw-r--r--"
        else
            type="файл"
            perms="-rw-r--r--"
        fi
    fi
done
```

- 3 Запустим скрипт (рис. 12):

```
asivanova1@fedora:~/scripts$ chmod +x myls.sh
asivanova1@fedora:~/scripts$ ./mysls.sh
Содержимое каталога: .
=====
-rwxr-xr-x backup_self.sh (исполняемый)
-rwxr-xr-x myls.sh (исполняемый)
-rwxr-xr-x print_args.sh (исполняемый)
asivanova1@fedora:~/scripts$
```

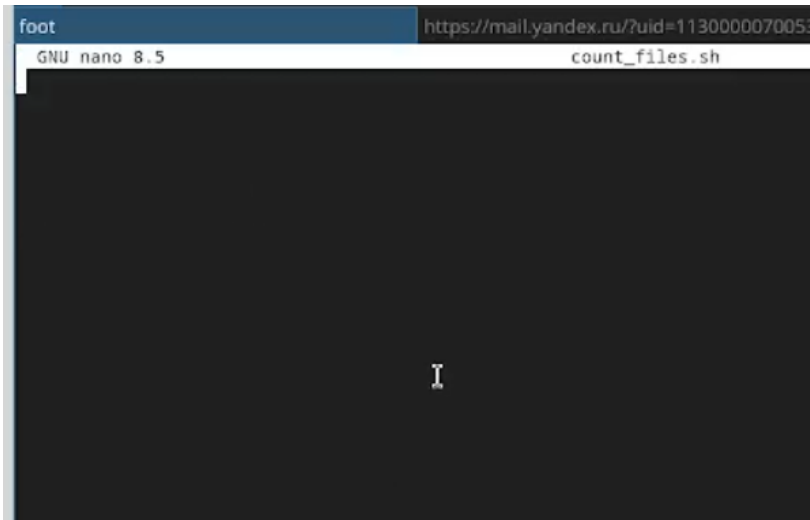
Рисунок 12: Запуск скрипта myls.sh

4 Запустим скрипт для каталога /etc (рис. 13):

```
-rw-r--r-- print_args.sh (исполняемый)
asivanova@fedora:~/scripts$ ./mys.sh /etc
Содержимое каталога: /etc
=====
drwxr-xr-x abrt (каталог)
-rw-r--r-- adjtime (файл)
-rw-r--r-- aliases (файл)
drwxr-xr-x alsa (каталог)
drwxr-xr-x alternatives (каталог)
drwxr-xr-x anaconda (каталог)
-rw-r--r-- anacrontab (файл)
-rw-r--r-- asound.conf (файл)
-rw-r--r-- at.deny (файл)
drwxr-xr-x audit (каталог)
drwxr-xr-x authselect (каталог)
drwxr-xr-x avahi (каталог)
drwxr-xr-x bash_completion.d (каталог)
-rw-r--r-- bashrc (файл)
-rw-r--r-- bindresvport.blacklist (файл)
drwxr-xr-x binfmt.d (каталог)
drwxr-xr-x bluetooth (каталог)
-rw-r--r-- chrony.conf (файл)
drwxr-xr-x cifs-utils (каталог)
drwxr-xr-x cockpit (каталог)
drwxr-xr-x credstore (каталог)
drwxr-xr-x credstore.encrypted (каталог)
```


Задание 4. Подсчёт файлов по расширению

- 1 Создадим файл скрипта `count_files.sh` (рис. 14):



2 Код скрипта (рис. 15):

```
foot https://mail.yandex.ru/?uid=1130000070053
GNU nano 8.5 count_files.sh
#!/bin/bash
# Задание 4: Подсчёт файлов по расширению
if [ $# -lt 2 ]; then
    echo "Использование: $0 <расширение> <директория>"
    echo "Пример: $0 txt /home/user"
    exit 1
fi
EXT="$1"
DIR="$2"
EXT="${EXT#*}"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: каталог $DIR не существует"
    exit 1
fi
COUNT=$(find "$DIR" -maxdepth 1 -type f -name ".*$EXT" 2>/dev/null | wc -l)
echo "Количество файлов с расширением .$EXT в каталоге $DIR: $COUNT"
```

3 Создадим тестовые файлы (рис. 16):

```
asivanova1@fedora:~$ cd ~/scripts/
asivanova1@fedora:~/scripts$ touch ~/test1.txt ~/test2.txt ~/test3.txt
asivanova1@fedora:~/scripts$ mkdir -p ~/testdir
asivanova1@fedora:~/scripts$ touch ~/testdir/file1.txt ~/testdir/file2.jpg
asivanova1@fedora:~/scripts$ ls
backup_self.sh count_files.sh myls.sh print_args.sh
asivanova1@fedora:~/scripts$ cd
asivanova1@fedora:~$ ls
1.cpp          Documents      'lab07.sh#'   scripts       test_mc       Изображения
abc1           Downloads     lab07.sh      ski.plases    work          Музыка
australia     feathers      LICENSE       test1.txt     work_1sem     Общедоступные
backup        file.txt      my_os         test2.txt     Видео         'Рабочий стол'
conf.txt      git-extended  play          test3.txt     Документы     Шаблоны
Desktop       GitHubDesktop-linux-amd64-3.4.7-linux1.deb  play_catalog  testdir       Загрузки
```

Рисунок 16: Создание тестовых файлов

- 4 Запустим скрипт (рис. 17):

```
asivanova1@fedora:~/scripts$ ./count_files.sh txt ~/
Количество файлов с расширением .txt в каталоге /home/asivanova1/: 2
asivanova1@fedora:~/scripts$ ./count_files.sh txt ~/scripts/
Количество файлов с расширением .txt в каталоге /home/asivanova1/scripts/: 3
asivanova1@fedora:~/scripts$
```

Рисунок 17: Запуск скрипта

- 5 Запустим скрипт для другого расширения (рис. 18):

```
Количество файлов с расширением .txt в каталоге /home/asivanova1/scripts/: 3
asivanova1@fedora:~/scripts$ ./count_files.sh jpg ~/scripts/testdir/
Количество файлов с расширением .jpg в каталоге /home/asivanova1/scripts/testdir/: 1
asivanova1@fedora:~/scripts$
```

Рисунок 18: Подсчёт файлов с расширением jpg

Раздел 4

4. Контрольные вопросы

4. Контрольные вопросы

1. Что такое командная оболочка? Приведите примеры.

Командная оболочка (shell) — программа для взаимодействия пользователя с ОС. Примеры: sh, bash, csh, ksh, zsh.

2. Что такое POSIX?

POSIX — набор стандартов для совместимости UNIX-подобных систем.

3. Как определяются переменные и массивы в bash?

Переменные: `var=value`. Массивы: `arr=(val1 val2)` или `set -A arr val1 val2`.

4. Каково назначение операторов let и read?

- let — арифметические вычисления

4. Каково назначение операторов let и read?

- let — арифметические вычисления
- read — чтение ввода с клавиатуры

5. Какие арифметические операции можно применять?

+, -, *, /, %, **, побитовые операции.

6. Что означает операция (())?

Двойные скобки для арифметических вычислений и условных выражений.

7. Какие стандартные имена переменных Вам известны?

PATH, HOME, USER, SHELL, PWD, IFS, MAIL, TERM, LOGNAME.

8. Что такое метасимволы?

Символы со специальным значением: *, ?, [,], >, <, |, &, ;, \$, \, `.

9. Как экранировать метасимволы?

- \ — экранирование одного символа

9. Как экранировать метасимволы?

- `\` — экранирование одного символа
- `'...'` — полное экранирование

9. Как экранировать метасимволы?

- `\` — экранирование одного символа
- `'...'` — полное экранирование
- `"..."` — частичное экранирование

10. Как создавать и запускать командные файлы?

Создать файл с `#!/bin/bash`, сделать исполняемым (`chmod +x`), запустить `./file.sh`.

11. Как определяются функции в bash?

```
function name { commands; }
```

или

```
name() { commands; }
```

12. Как выяснить, файл или каталог?

[-d "\$file"] — каталог, [-f "\$file"] — файл.

13. Назначение команд `set`, `typeset` и `unset`?

- `set` — установка параметров оболочки

13. Назначение команд `set`, `typeset` и `unset`?

- `set` — установка параметров оболочки
- `typeset` — атрибуты переменных

13. Назначение команд `set`, `typeset` и `unset`?

- `set` — установка параметров оболочки
- `typeset` — атрибуты переменных
- `unset` — удаление переменных/функций

14. Как передаются параметры в командные файлы?

Через позиционные параметры: \$1, \$2, ... \$9, \${10}.

15. Назовите специальные переменные `bash`.

`$0` — имя скрипта, `$#` — количество параметров, `$@` — все параметры, `$?` — код возврата, `$$` — PID, `$_` — PID последнего фонового процесса.

Раздел 5

5. Вывод

5. Вывод

Мы изучили основы программирования в оболочке `bash`, научились создавать командные файлы, работать с переменными и массивами, использовать управляющие конструкции `for`, `if`, организовывать поиск файлов и обрабатывать аргументы командной строки. В ходе работы были разработаны четыре скрипта: для создания резервной копии, обработки произвольного числа аргументов, аналог команды `ls` и подсчёт файлов по расширению.

